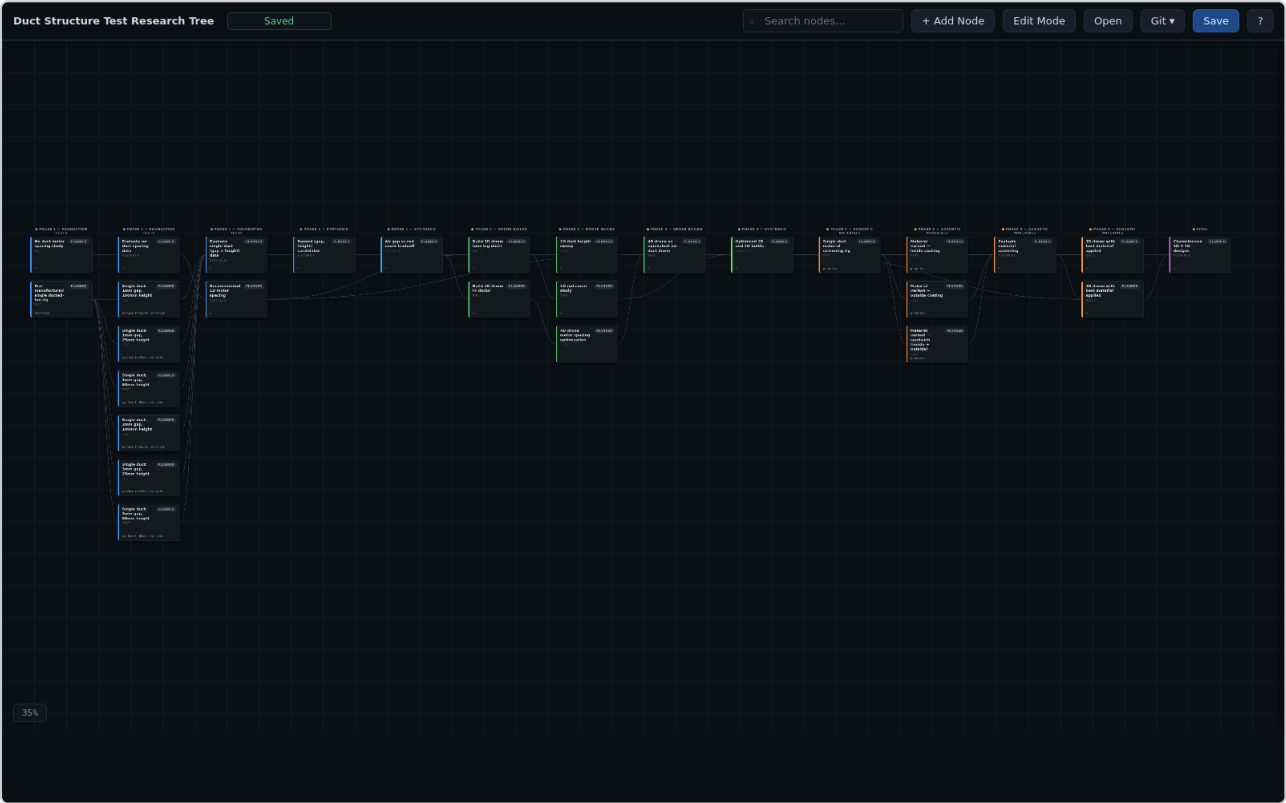


Research Tree Editor

User Manual — Shroud Comparison · Duct Structure Test Program

Version 2026-06-08 · single-file HTML editor + git-backed JSON

This document explains how to view, edit, and version-control the research tree that organises the Shroud Comparison test program. The audience is anyone joining the team mid-flight: no prior knowledge of the codebase is assumed. Read it once end-to-end (about twenty minutes); refer back to specific sections as needed.



The research tree at default zoom. Each column is a depth level; each card is one test, build, synthesis or decision node.

Table of contents

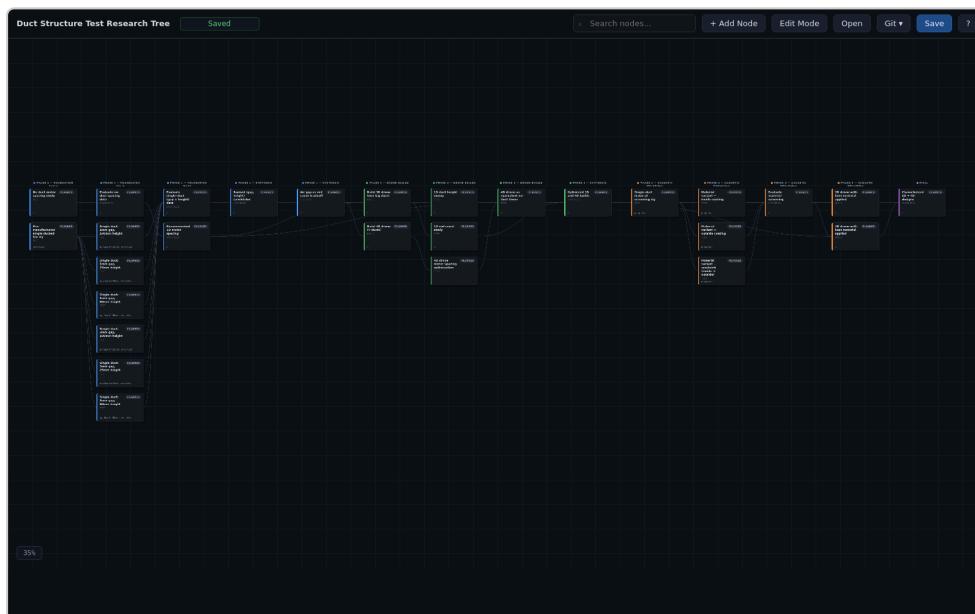
1	What this tool is
2	Getting started
2.1	Stopping & listing
3	The anatomy of a node
4	Editing a node
5	Adding & connecting nodes
5.1	Keyboard shortcuts
6	Saving
7	Git workflow
8	Searching
9	Worked example
10	Tree structure reference
11	Adding a new research tree
12	Troubleshooting / FAQ
13	Schema reference
14	Appendix — keyboard shortcuts

1. What this tool is

The research tree is a directed acyclic graph (DAG) that models the duct/shroud test program as a series of dependent steps. Each node is one experiment, one build, one synthesis, or one decision. Edges go left to right and express *this node depends on that one*.

Nodes carry only the metadata needed to plan and identify a test: phase, title, description, status, geometry (air gap, duct height, rod counts, weight, propeller size, motor spacing), notes, and a sound-visualizer link. The bulky data — far-field acoustic spectra, thrust curves, efficiency tables — lives in the separate sound visualizer project; this tree only points at it.

Durable history is provided by git. The single file `data.json` is the source of truth; every meaningful change should be committed. The editor's **Git** dropdown is just a friendly front-end over the real repository.



Tree overview. Columns are depth levels ($\text{column} = 1 + \text{max depth of parents}$); rows within a column are sorted by phase then alphabetically.

2. Getting started

From a terminal, launch the editor with the project launcher:

```
research_tree -duct
```

This starts a tiny Python server on `http://127.0.0.1:8123` and opens your default browser at that address. The server is bound to localhost only — it is not reachable from the network. The browser tab is the editor.

If you don't have the launcher on your `PATH`, you can run the server directly:

```
cd /home/adam/ŻYCIE/PRACA/Shroud_Comparison/research_tree
python3 serve.py --port 8123
```

Then point a Chromium-family browser at `http://127.0.0.1:8123/`.

2.1 Stopping & listing

Command	Effect
<code>research_tree -duct</code>	Start (or reuse) the server and open the browser.
<code>research_tree -duct --stop</code>	Stop the running server for this tree.
<code>research_tree --list</code>	List all registered trees, ports, and running state.

You can also just open `index.html` in a browser via the `file://` protocol — fine for read-only viewing, but the git buttons will be disabled because there's no server backing them.

3. The anatomy of a node

Every node on the canvas is a card of fixed size (220×130 px in CSS). Seven visible elements carry the meaning. The diagram below shows them on a scaled-up mockup of node `p1-ag1-h50` :



Phase color stripe. A 4 px wide bar on the left edge of the card, matching the phase color (blue Phase 1, green Phase 2, orange Phase 3, purple Final).

Title. Wraps fully inside the card (no truncation). Set in the side panel's *Title* field.

Status pill. Small rounded label top-right showing `PLANNED` , `IN-PROGRESS` , `DONE` , or `BLOCKED` .

Type label. One of `test` , `build` , `synthesis` , `decision` . Set in the side panel.

Geometry summary. Monospace one-liner of the non-null geometry fields, e.g.

```
ag=1mm h=50mm rods=t+b w=...g .
```

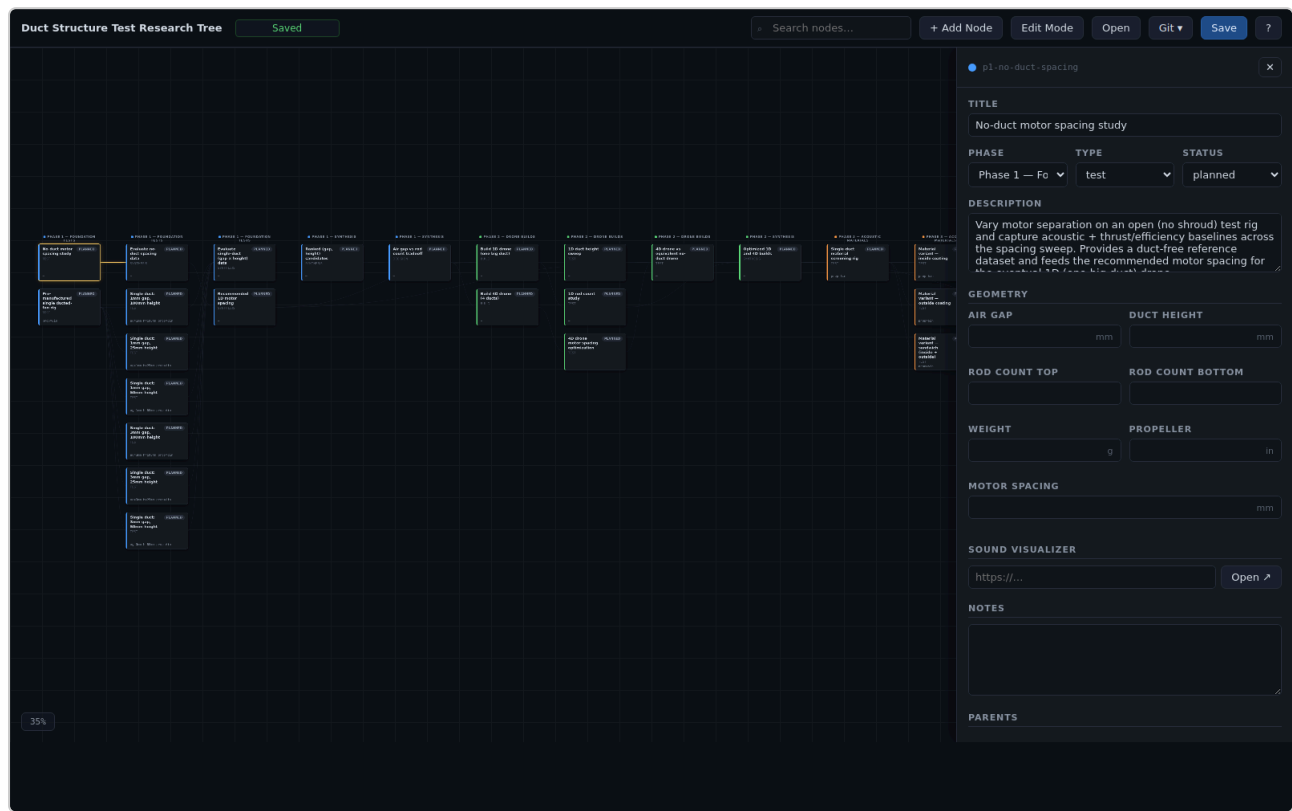
Link handle — small dot on the right edge of the card. Only visible in Edit Mode. Drag from this dot onto another node's body to make this node a parent of that node.

+ **Add-child handle** — small + button at the bottom-right corner. Only visible in Edit Mode. Click to create a child node off this one.

Tip. Elements 6 and 7 (link handle and add-child button) are only drawn on real cards when Edit Mode is on. The mockup above shows them so you can recognise them later. See *Section 5* for a real edit-mode screenshot.

4. Editing a node

Click any node on the canvas. A side panel slides in from the right with all the editable fields. Every field saves immediately into the in-memory state; the toolbar *Saved / Unsaved* pill turns yellow as soon as you change anything.



Side panel for `p1-no-duct-spacing` — a node with no geometry filled. Every geometry field is independently nullable.

4.1 The side-panel fields

Field	Meaning
Title	Short human label, used on the card and in search.
Phase	Drop-down. Determines the color stripe and the column header.
Type	test · build · synthesis · decision .
Status	planned · in-progress · done · blocked .
Description	1-3 sentences describing what the node represents.
Air gap (mm)	Tip-to-shroud air gap for the duct.
Duct height (mm)	Axial duct length.
Rod count top / bottom	Reinforcement rods on each side.
Weight (g)	Mass of the test article.

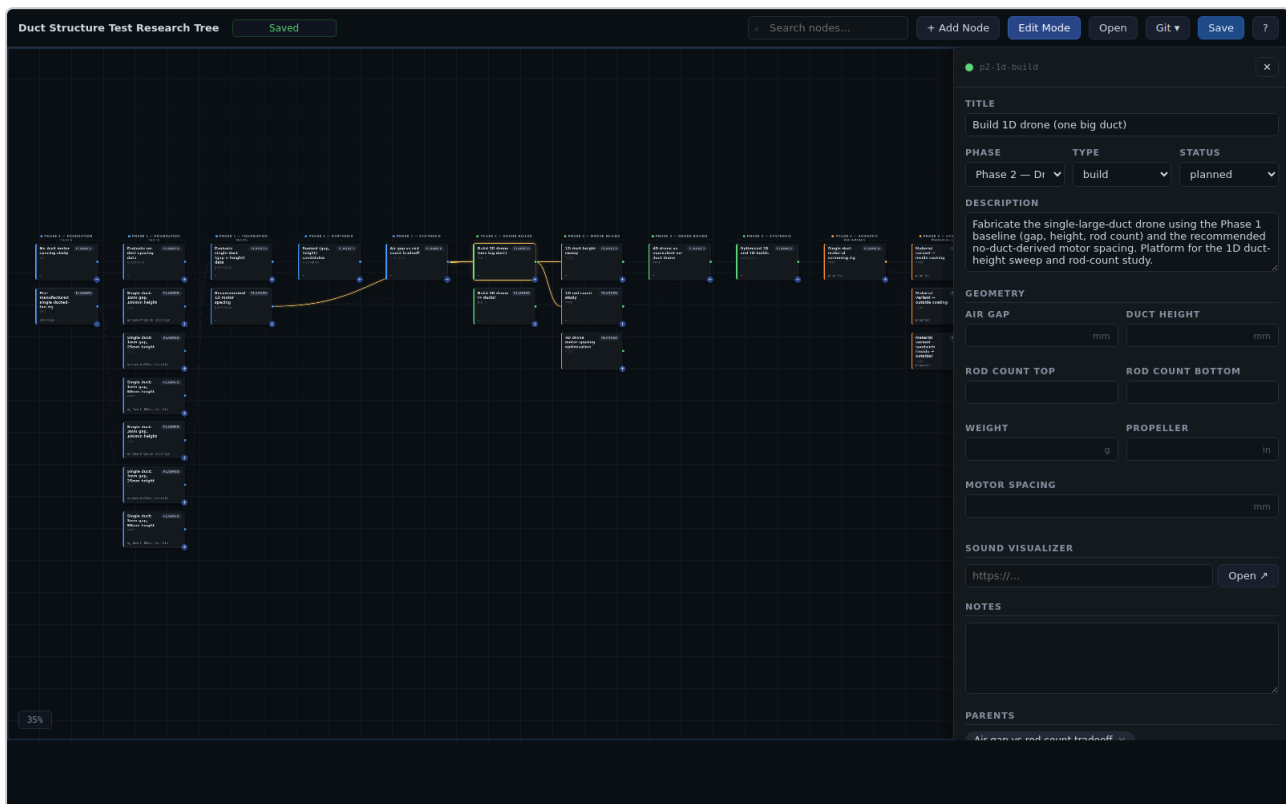
Field	Meaning
Propeller (in)	Propeller diameter in inches.
Motor spacing (mm)	Inter-motor distance for multi-motor builds.
Sound visualizer	URL pasted in from the separate sound visualizer tool. Click <i>Open</i> to deep-link out.
Notes	Free-form free-text scratchpad.
Parents	Chips listing this node's parents. Use the combobox to add more; click the × to remove.
Children	Read-only list of dependent nodes.

Empty geometry inputs save as `null` . That is expected for fields that don't apply to a particular node — for example a synthesis node has no air gap of its own. The geometry summary line on the card simply omits the null fields.

The bottom of the side panel has **Duplicate** and **Delete** buttons. Duplicate copies the node (new id, same parents, same geometry) and selects it. Delete asks for confirmation and removes both the node and any references to it from other nodes' `parents` lists.

5. Adding & connecting nodes

Most edits happen in **Edit Mode**. Click *Edit Mode* in the toolbar (or press the button at the top right) — the button highlights and the cards grow handles.



Edit Mode on. The button is highlighted (top right) and every card now sprouts a + handle at its bottom-right corner and a small link dot on its right edge.

5.1 Four ways to add a node

- **+ Add Node** on the toolbar — creates a free-floating node.
- **+ handle** on a card's bottom-right (Edit Mode) — creates a child of that node.
- **Right-click a node → Add child** — same effect.
- **Double-click an empty area of the canvas** (Edit Mode) — creates a node at that position.

5.2 Connecting nodes

- **Drag the link handle** (right-edge dot, Edit Mode) onto another node's body — makes this node a parent of the target. The connection is rejected with a toast if it would create a cycle.
- **Side-panel "+ Add parent" combobox** — pick a node from the list to add it as a parent of the current node.

5.3 Reordering

In Edit Mode, drag a card vertically within its column to change its row order. Across-column moves are determined automatically by the parent links — $\text{depth} = 1 + \text{max depth of parents}$, so dragging horizontally has no effect.

5.4 Keyboard shortcuts

Keys	Action
<code>Ctrl / Cmd + F</code>	Focus search box
<code>Ctrl / Cmd + S</code>	Save <code>data.json</code>
<code>Esc</code>	Close side panel / context menu / help overlay
<code>Delete</code>	Delete the selected node (with confirmation)

6. Saving

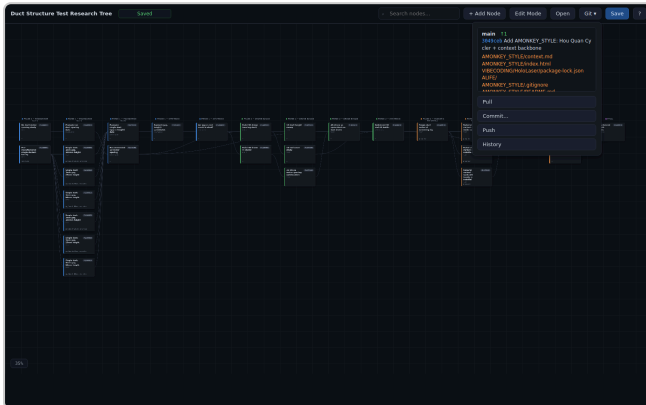
There are three save paths depending on how you opened the editor. The toolbar pill *Saved* turns into *Unsaved* the moment any field changes; saving turns it back to *Saved*.

Mode	What Save does
Over <code>http://</code> (launcher or <code>python3</code> <code>serve.py</code>)	Posts the JSON to the server, which writes <code>data.json</code> in place. No dialog, no permissions, no friction. This is the recommended path.
Over <code>file://</code> in Chromium (File System Access API)	The first save asks you to confirm a write back to <code>data.json</code> . From then on it writes silently to the same file.
Over <code>file://</code> in Firefox / Safari	The File System Access API is unavailable, so Save downloads <code>data.json</code> to your Downloads folder. You then have to copy it back into the <code>research_tree</code> directory by hand. Awkward — use the launcher instead.

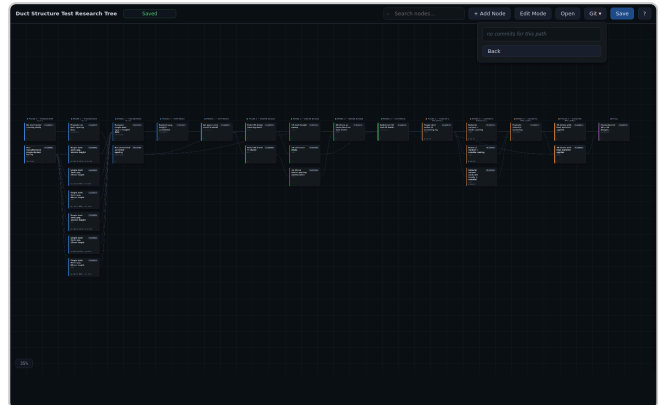
Watch out. The browser will warn you with the "Leave site?" dialog if you try to close the tab while changes are unsaved. Saving immediately or pressing `Ctrl` + `S` is the cheapest insurance.

7. Git workflow

The **Git** ▼ button in the toolbar is enabled only when you opened the editor over `http://` (i.e. via the launcher or `serve.py`). On `file://` the button is disabled.



Status panel: branch, ahead/behind, HEAD commit, dirty files.



History view: last 20 commits that touched this tree's `data.json`.

7.1 The four actions

Button	What it does
Pull	Runs <code>git pull</code> ; on success the editor re-fetches <code>data.json</code> so the canvas matches the new HEAD. If you have unsaved local changes, you'll be asked first — pulling discards them.
Commit...	Opens a one-line message box. Staging is automatic: only <code>data.json</code> is added. Hit <code>Enter</code> to commit, <code>Esc</code> to cancel.
Push	Runs <code>git push</code> . Errors (auth, conflict) surface as a toast.
History	Shows the last 20 commits that touched <i>this</i> <code>data.json</code> . <i>Back</i> returns to the status view.

History is real `git log`. You can also run it from a terminal: `git log -p PRACA/Shroud_Comparison/research_tree/data.json`. The per-edit history is intentionally not squashed — that history is the research log.

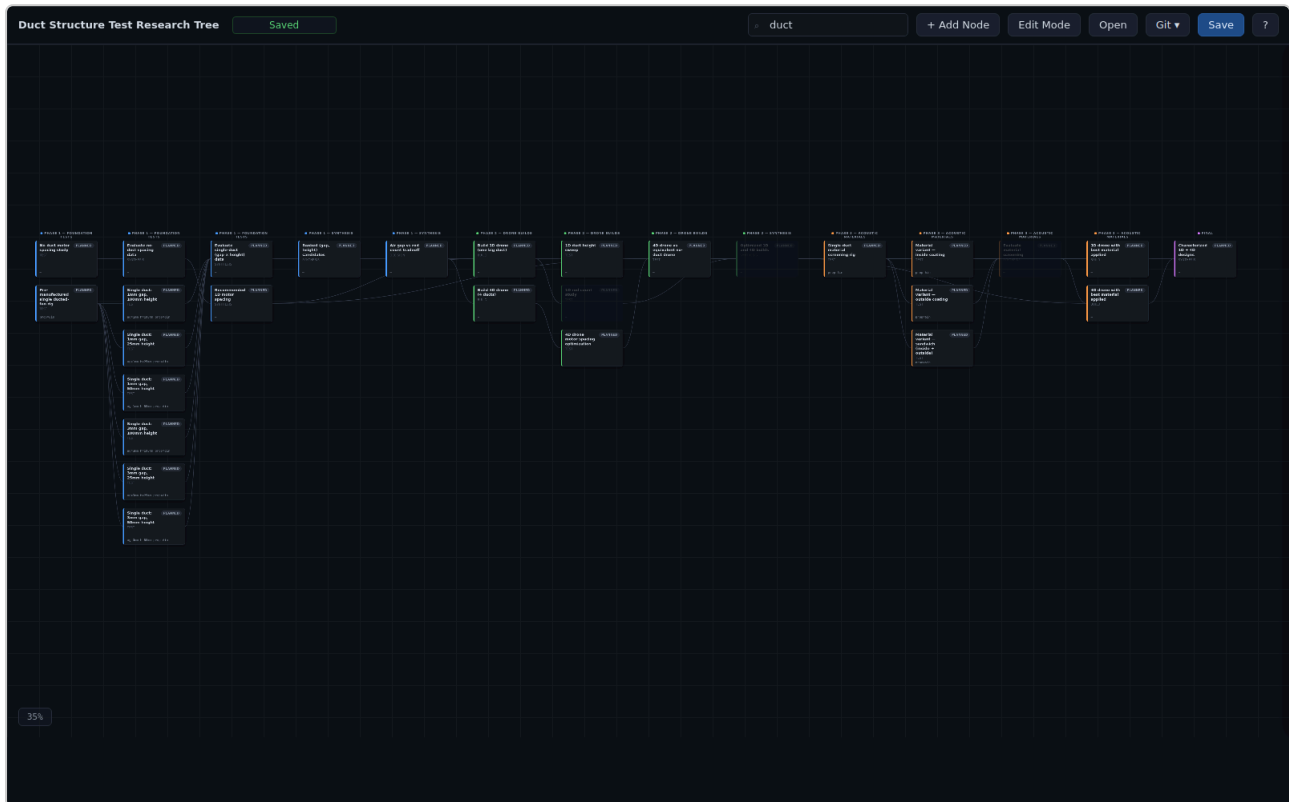
7.2 Suggested commit messages

Use a `tree:` prefix and describe what changed in a few words.

```
tree: fill ag1xh50 acoustic results
tree: split p2-ld-rod-count into 2/4/6/8 sub-runs
tree: mark p1-eval-no-duct as done
```

8. Searching

Type any word into the search box (or press **Ctrl** + **F**). Nodes whose title or description contains the term remain bright; the rest are dimmed. The connections stay visible so you can still see the surrounding structure. Press **Esc** while the search input is focused — or simply delete the term — to clear the filter.



Filtering the tree on the term `duct` . Matches stay bright, non-matches fade.

9. Worked example

Suppose you have just finished bench-testing the single-duct rig with a 1 mm air gap and a 50 mm duct height. Acoustic + thrust numbers live in the sound visualizer; you want to capture the geometry & status here and commit.

1. From a terminal: `research_tree -duct` . The browser opens.
2. On the canvas, click the card titled **Single duct: 1mm gap, 50mm height**. The side panel opens (see Section 3 screenshot).
3. The **Air gap** and **Duct height** fields are already filled (`1` and `50`). Type the measured **Weight** in grams.
4. In the **Sound visualizer** field, paste the URL from the visualizer's session page.
5. Type a one-line summary in **Notes**: e.g. "first run, sweep T1 → T6 captured 2026-06-08".
6. Change **Status** from `planned` to `done` .
7. Click **Save** (or press `Ctrl` + `S`). The *Saved* pill turns green.
8. Click **Git ▼**, then **Commit...**, type:
`tree: ag1xh50 results from 2026-06-08`
and press `Enter` .
9. Click **Push**. Done — the change is on the remote, with a clean diff against the previous state.

10. Tree structure reference

10.1 Phases

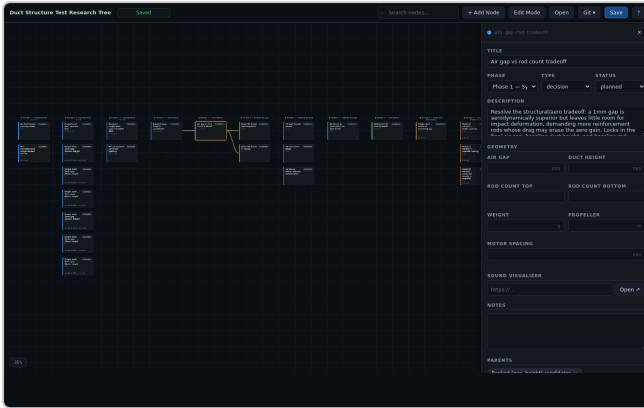
id	Title	color
phase1	Phase 1 — Foundation Tests	#4a9eff (blue)
phase1_synth	Phase 1 — Synthesis	#4a9eff (blue)
phase2	Phase 2 — Drone Builds	#5cd97a (green)
phase2_synth	Phase 2 — Synthesis	#5cd97a (green)
phase3	Phase 3 — Acoustic Materials	#ff9c4a (orange)
final	Final	#d97aff (purple)

10.2 Node types

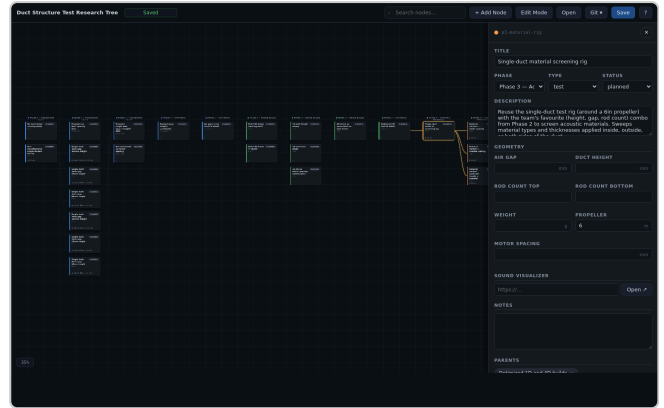
type	Meaning
test	A physical experiment on a rig or drone. Geometry fields usually filled.
build	A fabrication step — assembling a 1D or 4D drone or sub-assembly.
synthesis	Aggregation/evaluation node. Combines several test outputs into a recommendation.
decision	Branch point: the team chooses a configuration that locks downstream nodes.

10.3 Statuses

status	Meaning
planned	On the roadmap, not started.
in-progress	Active work.
done	Complete; results captured (link & notes filled).
blocked	Cannot proceed until something upstream resolves.



A decision node (*pls-gap-rod-tradeoff*) — Phase 1 air-gap vs rod-count tradeoff.



Phase 3 root: the single-duct material screening rig that feeds the inside/outside/sandwich material variants.

11. Adding a new research tree

The `research_tree` bash launcher uses a heredoc table called `TREES` to map a flag (like `-duct`) to a directory + port. To register a second tree:

1. Create a new folder, e.g.

```
/home/adam/ŻYCIE/PRACA/SomeProject/research_tree/
```

2. Copy `index.html`, `serve.py` and a starter `data.json` into it. The schema must match (see Section 13).

3. Edit the `research_tree` launcher, find the `TREES` heredoc, and append one row:

```
-myproj /home/adam/ŻYCIE/PRACA/SomeProject/research_tree 8125
```

4. Pick a port that doesn't clash with any other tree. Each tree gets its own port and its own server process.
5. Verify with `research_tree --list`. Then `research_tree -myproj` opens the new tree.

Each tree's `serve.py` is fully self-contained: it resolves the enclosing git repository on startup and exposes `/api/git/*` rooted there. So the same launcher pattern works for any tree whose `data.json` sits in a git repo.

12. Troubleshooting / FAQ

Symptom	Fix
Git buttons are disabled / greyed out.	You opened the editor via <code>file://</code> . The git endpoints only exist over <code>http://</code> . Close the tab and run <code>research_tree -duct</code> (or <code>python3 serve.py</code>) instead.
Browser opens twice when I run the launcher.	Used to happen on first start + reload; doesn't anymore — the launcher only opens a browser when it actually starts a new server. If you still see this, an older copy of the launcher is on your PATH.
Save downloaded the file instead of saving in place.	You're on Firefox/Safari over <code>file://</code> . The File System Access API isn't available there. Use the launcher (server mode) or switch to Chromium.
Pull failed with a merge conflict.	The server refuses to auto-resolve. Drop to a terminal in the repo, run <code>git status</code> , resolve manually (most often by accepting one side of <code>data.json</code>), then commit.
Long node titles are getting cut off.	Not anymore — titles wrap fully inside the 220×130 card. If you see ellipsis, hard-refresh the page (<code>Ctrl + Shift + R</code>) to pull the latest <code>index.html</code> .
The canvas is empty after launch.	The fetch for <code>data.json</code> failed. Check the browser console; usually it means the file is malformed or missing. Restore from git: <code>git checkout -- data.json</code> .
Adding a parent says "would create a cycle".	The DAG invariant is enforced. Either the target node is already a descendant, or you've picked the wrong direction. Cancel and try the inverse link.

13. Schema reference

The on-disk `data.json` shape (must stay stable — both the editor and any future tooling depend on it):

```
{
  "title": "Duct Structure Test Research Tree",
  "phases": [
    {"id": "phase1",          "title": "Phase 1 – Foundation Tests",    "color": "#4a9eff"},
    {"id": "phase1_synth",   "title": "Phase 1 – Synthesis",          "color": "#4a9eff"},
    {"id": "phase2",          "title": "Phase 2 – Drone Builds",        "color": "#5cd97a"},
    {"id": "phase2_synth",   "title": "Phase 2 – Synthesis",          "color": "#5cd97a"},
    {"id": "phase3",          "title": "Phase 3 – Acoustic Materials", "color": "#ff9c4a"},
    {"id": "final",           "title": "Final",                    "color": "#d97aff"}
  ],
  "nodes": [
    {
      "id": "kebab-slug",
      "phaseId": "phase1",
      "title": "Short title",
      "description": "1–3 sentences.",
      "type": "test | build | synthesis | decision",
      "status": "planned | in-progress | done | blocked",
      "parents": ["other-id"],
      "geometry": {
        "airGapMm": null, "ductHeightMm": null,
        "rodCountTop": null, "rodCountBottom": null,
        "weightG": null, "propellerInches": null, "motorSpacingMm": null
      },
      "soundVisualizerLink": "",
      "notes": ""
    }
  ]
}
```

Rules

- kebab-case unique IDs.
- `parents` references IDs only (no orphans).
- All geometry keys present (use `null` for N/A).
- New nodes start `"planned"` with empty `soundVisualizerLink` and `notes`.
- No cycles in `parents` — the editor refuses to create one.
- Layout is implicit: $\text{column} = 1 + \text{max depth of parents}$. Structure is purely a function of the parent links.

14. Appendix — keyboard shortcuts

Keys	Action	Where
Ctrl / Cmd + S	Save data.json	Anywhere
Ctrl / Cmd + F	Focus the search box	Anywhere
Esc	Close side panel	When side panel is open
Esc	Close context menu	When right-click menu is open
Esc	Close Help overlay	When Help is open
Esc	Cancel commit form	In the commit prompt
Delete / Backspace	Delete the selected node (with confirm)	Selection on canvas, not in an input
Enter	Submit commit message	In the commit prompt
Click node	Select & open side panel	Canvas
Drag empty	Pan the canvas	Canvas background
Wheel	Zoom (anchored on cursor)	Canvas
Right-click node	Context menu: Duplicate, Add child, Delete	Canvas
Double-click empty	Create node here	Canvas (Edit Mode only)
Drag node	Reorder within column	Canvas (Edit Mode only)
Drag right-edge dot	Link this node as parent of the target	Canvas (Edit Mode only)

URL hash hooks (for deterministic screenshots & deep links)

hash	Effect
#node=<id>	Select that node and open the side panel.
#edit	Enable Edit Mode.
#git	Open the Git dropdown panel (server mode only).
#git=history	Open Git panel and switch to History view.
#help	Open the Help overlay.
#search=<text>	Pre-fill the search box and apply the filter.

Hooks combine with `&`, e.g. `#node=p1-ag1-h50&edit`. `hashchange` events re-apply the state without a reload.